

DATABASE DESIGN AND TECHNOLOGY SURVEY.

PHASE 1

NAME: KURALOVIYAN SENTHILNATHAN

STUDENT ID: kms7653

CMPSC 431W

TABLE OF CONTENTS

1) REQUIREMENT ANALYSIS	03
TABLE ATTRIBUTES.....	03
FUNCTIONALITIES.....	07
2) CONCEPTUAL DATABASE DESIGN.....	16
3) TECHNOLOGY SURVEY.....	21
4) LOGICAL DATABASE DESIGN AND NORMALIZATION....	30
SCHEMA	33
5)REFERENCES.....	36

TASK 1:

REQUIREMENT ANALYSIS

TABLES ATTRIBUTES:

1)APPLICATION_USERS

USER_ID: It is a registered CUSTOMER_ID or MANAGER_ID users can use to login and use the functionalities.

PASSWORD: it is the password customers/managers can use to login into their account.

PRIVILEGE: If it is a manager that is trying to log in, PRIVILEGE should be 1, If it is a customer that is trying to log in PRIVILEGE = 0

1)BOOK DATA

ISBN, TITLE, AUTHOR, PUBLISHER, LANGUAGE, PUBLISHING_DATE, EDITION_NUMBER, SIZE, PRICE, KEYWORD, SUBJECT, NO_PURCHASED_BOOKS, NO_SOLD_BOOKS

Here,

ISBN: It is the International Standard Book Number which is assigned to each separate edition and variation of a publication. Therefore, it is the Primary key of the table.

TITLE: it refers to the title of the book. (book name)

AUTHOR: It refers to the Primary author name

PUBLISHER: the person or company that issued the book.

LANGUAGE: It is used to identify which language the book was published in.

PUBLISHING_DATE: Date of publication.

EDITION_NUMBER: It contains a numerical value which refers to which edition or variation of publication.

SIZE: Number of pages in the book

PRICE: Cost of the book in U.S dollars currency

KEYWORD: Special words associated with the book.

SUBJECT: It refers to the book genre:

- (a) Action and Adventure
- (b) Classics
- (c) Comic Book or Graphic Novel

- (d) Detective and Mystery
- (e) Fantasy
- (f) Historical Fiction
- (g) Horror
- (h) Science Fiction
- (i) Romance

NO_PURCHASED_BOOKS: It refers to the total number of books purchased by the bookstore or ever delivered by the supplier

NO_SOLD_BOOKS: It refers to the total number of books ever sold to the customers by the bookstore.

We can find the stock available for any book by simply subtracting:

$(NO_PURCHASED_BOOKS) - (NO_SOLD_BOOKS) = \text{Stock for a certain book}$

2)CUSTOMER_DATA

FIRST_NAME, LAST_NAME, CUSTOMER_ID, ROLE, ADDRESS, MOBILE_NUMBER, EMAIL_ID, BOOKS_ORDERED

Here,

FIRST_NAME: Customer's First name

LAST_NAME: Customer's Last name

CUSTOMER_ID: It is the login ID/username that customers can make to register and use to find information on their orders. It is the Primary Key for the Table and it is also a foreign key to the TRANSACTION HEADER TABLE.

ADDRESS: The customer's home address to deliver the books.

MOBILE_NUMBER: Mobile number of the customers to send information on their orders and provide notifications on newly arrived books and other suggested books.

EMAIL_ID: email address of the customers to send information on their orders and provide notifications on newly arrived books and other suggested books.

BOOK_ORDERED: the number of books the customer has currently ordered.

3)TRANSACTION HEADER TABLE

CUSTOMER_ID, TRANSACTION_ID, TRANSACTION_DATE, ISBN, QUANTITY, TRANSACTION_STATUS, FINAL_AMOUNT

CUSTOMER_ID: the login ID that customers use to place an order. It is a foreign key for this table.

TRANSACTION_ID: Unique number to identify the specific transaction.

ISBN: It is the International Standard Book Number for which a user wishes to buy. It is a foreign key for this table.

TRANSACTION_DATE: Date of transaction

QUANTITY: The number of books the user wishes to buy

TRANSACTION_STATUS: It refers to whether the transaction is new, processing or closed.

PRICE: It is the individual price of the book at the time of order.

AMOUNT: It is the price of an individual book multiplied with its quantity ordered

Here, TRANSACTION_ID and ISBN together form the Primary key for this table.

4)COMMENTS

ISBN, CUSTOMER_ID, COMMENT_ID, BOOK_SCORE, COMMENT_TEXT, USEFULNESS_RATINGS, USER_LEGITAMACY, FEEDBACK_DATE.

ISBN: It is the International Standard Book Number for which a user wishes to make a comment.

CUSTOMER_ID: It is the customer ID of a user wishes to make a comment.

COMMENT_ID = It is a unique number used to identify a comment. It is the primary key in the table.

BOOK_SCORE: It is a numerical score provided by the users for a particular book.

It can range from 0 to 10 where:

0 = terrible

2 = Not very good

4 = Average

6 = Not bad

8 = Good

10 = Masterpiece

TEXT: It is an optional short text user can type as their comments.

USEFULNESS_RATINGS: Users can view a comment and provide a usefulness score on three levels:

1 = Useless

2 = Useful

3 = Very Useful

USER_LEGITAMACY: A user can declare any other user as trusted or not trusted.

FEEDBACK_DATE: The date a comment was made by a particular user.

COMMENT_ID is the primary key for the table.

CUSTOMER_ID and ISBN are foreign keys.

5)MANAGER

FIRST_NAME, LAST_NAME, MANAGER_ID, MOBILE_NUMBER, EMAIL_ID

FIRST_NAME: Manager's First name

LAST_NAME: Manager's Last name

MANAGER_ID: It is the login ID that managers can make for other managers to register and use to find information on their customer's orders. It is the Primary Key for the Table.

MOBILE_NUMBER: Mobile number of the managers to send notifications on newly arrived books.

EMAIL_ID: email address of the managers to provide notifications on newly arrived books.

6)DELIVERY_DETAILS TABLE

RECORD_NUMBER, RECEIVED_DATE, MANAGER_ID, ISBN, NEW_STOCK, SUPPLIER

RECORD_NUMBER: Unique number to keep track of all deliveries to the warehouse.

RECEIVED_DATE: It is the date a particular delivery has been made and recorded.

MANAGER_ID: It is the manager ID of the manager present when the delivery has occurred and been recorded by the manager. It is a foreign key.

ISBN: It is the International Standard Book Number of the book that was delivered.

NEW_STOCK: The number of books that were delivered for each ISBN.

SUPPLIER: It is the name of the person or company that supplied the books to the warehouse.

RECORD_NUMBER and ISBN together form the primary key for the table.

FUNCTIONALITIES:

1)Customer Registration:

Here the client (participating party) is the user.

A new user has to provide the appropriate information as specified in the customer data table. He/she can pick a login name and a password.

Hence, the inputs are FIRST_NAME, LAST_NAME, CUSTOMER_ID, PASSWORD, ADDRESS, MOBILE_NUMBER, EMAIL_ID.

The output refers to whether the registration was successful or not.

The CUSTOMER_ID and PASSWORD are stored in the the APPLICATION_USERS table and the PRIVILEGE is set to 0. The FIRST_NAME, LAST_NAME, CUSTOMER_ID, PASSWORD, ADDRESS, MOBILE_NUMBER, EMAIL_ID attributes are stored in the CUSTOMER_DATA table.

Constraints:

Login names of the users must be unique to each other and are case insensitive. They should not contain any special characters and should not exceed a MINIMUM length of 5 characters and a MAXIMUM length of 8 characters.

Password of the user must have a minimum length of 10 characters and must contain at least 1 special character, 1 uppercase letter and 2 or more numbers and maximum length of 30.

Mobile number and email address must be provided in order to send notifications to users about their orders.

The function first checks if the login names the customer provides when registering exists in the CUSTOMER table. If it does not exist, then it is considered unique and they can process to setting up the password. This is will increase the number of customers in the CUSTOMER table.

2) Multiple Managers:

Here the client (participating party) is the manager.

There can be multiple users who can access the full functionality of the system.

When a new manager is registered into the system, the number of managers increase in the MANAGER table and the function takes in details like FIRST_NAME, LAST_NAME, MANAGER_ID, PASSWORD, MOBILE_NUMBER, EMAIL_ID.

Only a manager can register a new manager to the system.

There should be a superuser being the first manager when the system starts for the first time. Therefore, only a manager can register a new manager into the MANAGER table. Only a manager can access the DELIVERY_DETAILS_HEADER TABLE to record the different deliveries made to the warehouse and customers cannot access this table. When a manager is being registered by another manager, the PRIVILEGE attribute in APPLICATION USER table is internally set to 1 by program.

The output of the function is if registration is successful and is based on whether the MANAGER_ID selected on registration is unique among all the MANAGER_ID and CUSTOMER_ID and the password constraints are met.

Login names of the users must be unique to each other and are case insensitive. They should not contain any special characters and should not exceed a MINIMUM length of 5 characters and a MAXIMUM length of 8 characters.

Password of the user must have a minimum length of 10 characters and must contain at least 1 special character, 1 uppercase letter and 2 or more numbers and maximum length of 30.

3)Ordering:

Here the client (participating party) is the user.

After registration, a new user can order one or more books by placing an order in this function.

The input to this function will be ISBN, CUSTOMER_ID, QUANTITY. The user can order multiple copies of a book one or more times.

The output is the total amount of its order reported to him/her, the TRANSACTION_ID and TRANSACTION_STATUS.

When an order is successfully made, a new TRANSACTION_ID is created to uniquely identify the order and the TRANSACTION_STATUS becomes available and It refers to whether the transaction is new, processing or closed.

The function first checks if the book the user is ordering is available in stock or not. In order to do this, it navigates to the BOOK_DATA table and finds the total stock of the book by calculating:

$(NO_PURCHASED_BOOKS) - (NO_SOLD_BOOKS) = \text{Stock for a certain book.}$

If there is stock available for the quantity of the book the user wishes to order, the TRANSACTION_STATUS becomes "processing". If there is less or no stock available for the quantity of books the user ordered, the TRANSACTION_STATUS becomes "new".

4)New book:

Here the client (participating party) is the manager.

The inputs are ISBN, TITLE, AUTHOR, PUBLISHER, LANGUAGE, PUBLISHING_DATE, EDITION_NUMBER, SIZE, PRICE, KEYWORD, SUBJECT, NO_PURCHASED_BOOKS, NO_SOLD_BOOKS

The store managers may record the details of a new book and the number of books that have arrived in the warehouse.

The output is whether the new book added is successful or not.

After a delivery of books has been made by a supplier to the warehouse and the store managers provides the inputs, the function adds the new TITLE and the NO_PURCHASED_BOOKS in the BOOK DATA table along with the other attributes. The function adds the quantity of the particular book received, i.e., by adding NEW_STOCK from DELIVERY_DETAILS_HEADER TABLE to the NO_PURCHASED_BOOKS in the BOOK DATA table.

5. Arrival of more copies:

Here the client (participating party) is the manager.

The store manager log in with their MANAGER_ID and record the date of delivery which is stored in RECEIVED_DATE, the ISBN of the books that were delivered, quantity of each book delivered is stored in NEW_STOCK and the SUPPLIER.

OUTPUT:

A unique record number is generated for each delivery of books and is stored in RECORD_NUMBER in the DELIVERY_DETAILS_HEADER TABLE.

Only a manager can access the DELIVERY_DETAILS_HEADER TABLE to record the different deliveries made to the warehouse and customers cannot access this table.

Only 1 manager has to be present when a delivery or resupply of books occur in the warehouse.

The store managers change the stock level of a given book in the BOOK_DATA table.

After a delivery of books has been made by a supplier to the warehouse, the store managers provides the inputs and the function updates the NO_PURCHASED_BOOKS in the BOOK DATA table by adding the quantity of the particular book received, i.e., by adding NEW_STOCK from DELIVERY_DETAILS_HEADER TABLE to the NO_PURCHASED_BOOKS in the BOOK DATA table.

6)Comments:

Here the client (participating party) is the user.

The inputs are ISBN, CUSTOMER_ID, BOOK_SCORE, TEXT, FEEDBACK_DATE

The COMMENT_ID is generated and acts as the primary key for the COMMENT table and can be used to uniquely identify every comment.

A comment can only be changed by the customer who made it.

Only one comment per user per book is allowed.

The user is allowed to give an optional text which cannot cross 100 characters.

The numerical score provided by the user should be between 0-10 where:

0 = terrible

2 = Not very good

4 = Average

6 = Not bad

8 = Good

10 = Masterpiece

Once a comment is made, the ISBN, CUSTOMER_ID, BOOK_SCORE, TEXT and FEEDBACK_DATE are stored in the COMMENT table.

The output of this function is a new comment stored in a text box.

7. Usefulness ratings:

Here the client (participating party) is the user.

In this function, a user can view/access any customer's comment on any book/ISBN and provide a usefulness rating on that particular comment.

Users can access a comment and give it a usefulness score in three levels:

- (a) Useless = 0
- (b) Useful = 1
- (c) very useful = 2

A user should not be allowed to provide a usefulness rating for his/her own comments.

A user can provide a usefulness rating only once for a particular customer's comment but is allowed to change his/her usefulness rating for the particular customer's comment.

This is stored in the USEFULNESS_RATINGS attribute of the COMMENT table for every customer.

A side effect for this function is an increase the usefulness ratings for the customer's comment.

8) Trust recordings:

Here the client (participating party) is the user.

In this function, a user can view/access any customer's comment on any book/ISBN and provide a trust rating for that customer.

The input for this function is either:

0 = Not trusted
1 = Trusted.

The trust recording is stored in the USER_LEGITAMACY attribute in the COMMENT table. It is stored as 0 if Not trusted and 1 as Trusted.

A user may declare zero or more other users as trusted or not trusted.

A user cannot declare a trust/not trusted rating for his/her own comments.

This is stored in the USER_LEGITAMACY attribute of the COMMENT table for every customer.

9)Book browsing:

Here the client (participating party) is the user.

The inputs can be AUTHOR, PUBLISHER, KEYWORD, LANGUAGE, SUBJECT and the sorting order.

The sorting order can be as follows:

- (a) by publish date
- (b) by the average numerical score of the comments
- (c) by the average numerical score of the trusted user feedbacks.

A minimum of one input among AUTHOR, PUBLISHER, KEYWORD, LANGUAGE, SUBJECT is necessary, and the sorting order is also necessary in order to search based on the category and It is searched using the BOOK DATA table.

The output will be the ISBN, BOOK_NAME from BOOK_DATA table of the list of books that fall under the same category

We can sort results on publish date by ordering the PUBLISHING_DATE data in the BOOK data table in descending order.

We can also sort the average numerical score of the comments using the BOOK_SCORE

We can also sort the average numerical score of the trusted users by first calculating the average trusted score for each user from the USER_LEGITAMACY attribute in the comments table and then ordering them in descending order.

There will be no Side effects to any of the data or tables when this function is used.

10)Useful Comments:

Here the client (participating party) is the user.

The inputs are ISBN and n,
here n denotes the top “n” useful comments.

For a given book, a customer could ask for the top “n” most useful comments.
The value n is user-specified (say 5 or 10). The value of n should not cross 20.
The usefulness of a comment can be calculated using its average usefulness score.

There will be no side effects to the data stored in any of the tables and the output of the function would return the top “n” comments.

Once the user enters the ISBN number and n value, the function calculates the average usefulness score of every comment for the particular book which is present in the COMMENT table and returns the top “n” comments in descending order.

11) Buying suggestions:

Like most e-commerce websites, when a user orders a copy of book A, the system gives a list of other suggested books. Book B is suggested if there is another user that bought both A and B, else another book is suggested if it falls in the same subject and language category.

The suggested books is sorted on the most quantity sold for each book (i.e., most popular first) among the users who also bought A.

This is achieved by using the TRANSACTION_HEADER TABLE. The function will first go through the transaction orders to find if BOOK A was bought by another customer along with another book B. When found, it will add the book ISBN to the list of other ISBNs of books found in other transactions where book A was also bought. Once we have the list of books to suggest, we sort them using the NO_SOLD_BOOKS attribute from the BOOK DATA table in descending order(decreasing sales amount).We then provide this final sorted list of suggested books when BOOK A is purchased by a customer.

12) Degrees of separation:

Given an author X, a user may want to know the books written by authors who are 1-degree and 2-degree separated with X. The degree of separation is defined as follows: two authors A and B are 1-degree away if they co-authored at least one book together; they are 2-degrees away if there exists an author C who is 1-degree away from each of A and B AND A and B are not 1-degree way at the same time.

13) Book Statistics:

Here the client (participating party) is the manager.

Every semester, the store manager would want:

(a) The list of m most popular authors (in terms of copies sold this quarter)

- (b) The list of m most popular publishers (in terms of copies sold this quarter)
- (c) The list of m (say m = 10) most popular books (in terms of copies sold this quarter)

The input for this function would be the which option the manager prefers.

The output will be the list according to the chosen option.

For finding the top “m” most popular authors, we only have to order/arrange the NO_SOLD_BOOKS attribute in the BOOK DATA table in descending order for the particular period of the year, which can be found in and then list the corresponding AUTHOR attributes list the popular author names.

For finding the list of “m” most popular books, we use the TRANSACTION HEADER TABLE. The function counts the quantity of each ISBN of the book ordered in the quarter by checking the TRANSACTION_DATE.

We then arrange the count of each ordered book quantity in descending order. This would give the list of “m” most popular books. After finding the list of “m” most popular books, we can use the respective ISBN’s of the books and use the BOOK DATA table to find their respective AUTHOR and PUBLISHER attributes to provide the list of “m” most popular authors and list of “m” most popular publishers for the quarter respectively.

14. User awards:

Here the client (participating party) is the manager.

The input is M , an integer value.

The store manager also wants to give awards to the “best” users; Thus, the manager needs to know:

- The top m most trusted users (the trust score of a user is the count of users trusting him/her minus the count of users not-trusting him/her)

We can calculate this by getting the average trust score, using the USER_LEGITAMACY attribute of the user from the COMMENT table.

- The top m most useful users (the usefulness score of a user is the average usefulness of all his/her feedbacks combined).

We can calculate this by getting the average usefulness score, using the usefulness_score attribute of the user from the COMMENT table.

There will be no side effect and the output is the the top m most trusted users and the top m most useful users

16)Most valuable customer.

Here the client (participating party) is the manager.

In this function, the manager can find the most valuable customer for a particular time period.

The input for this function will be the time period and the function will use the TRANSACTION_HEADER table to and use the TRANSACTION_DATES attribute for highlighting the particular time period and then finding the CUSTOMER_ID with the maximum value for the AMOUNT attribute.

The output of the function will be the CUSTOMER_ID.

There will be no side effect when the function is used.

17)Restock

Here the client (participating party) is the manager.

In this function, the manager can find whether any books are out of stock. There will be no input and the function would use the BOOK_DATA table. The function would check if the NO_PURCHASED_BOOKS and NO_SOLD_BOOKS attributes are equal and if they are equal, then the book is out of stock and would alert the manager.

The output of the function would be the ISBN of all the books out of stock.

There will be no side effect when the function is used.

18)delete book

Here the client (participating party) is the manager.

When a book is no longer being published, and the bookstore is out of stock for the certain book, the manager can use this function to delete the book details from the BOOK_DATA table.

The inputs for this function is the ISBN of the book.

The output of the function will state whether deletion was successful or if still certain stock of the book is available.

The function will find the respective ISBN and delete the entire row in the BOOK_DATA table.

19)Transaction history

Here the client (participating party) is the user.

In this function, the user is allowed to view their transaction history for the previous quarter.

The inputs for this function are CUSTOMER_ID, PASSWORD.

The output of this function would be the TRANSACTION_ID, TRANSACTION_DATE, ISBN, QUANTITY, TRANSACTION_STATUS, FINAL_AMOUNT.

The function uses the data present in the attributes of TRANSACTION_HEADER table and list the details according to specific CUSTOMER_ID.

The constraints are that the function requires the customer's password and will show the transaction history only for the previous quarter

There will be no side effect when the function is used.

20)new arrivals.

Here the client (participating party) is the user.

Based on the DELIVERY_DETAILS TABLE, we can see the date of the newly arrived books and based on the edition number, where its value should be 1. The edition_number attribute is found in the BOOK_DATA table.

The function will show the top 10 new arrival books as output and there is no side effect.

TASK 2:**CONCEPTUAL DATABASE DESIGN**

Based on the result of Task 1, Below is present an entity-relationship (ER) diagram describing the conceptual database design.

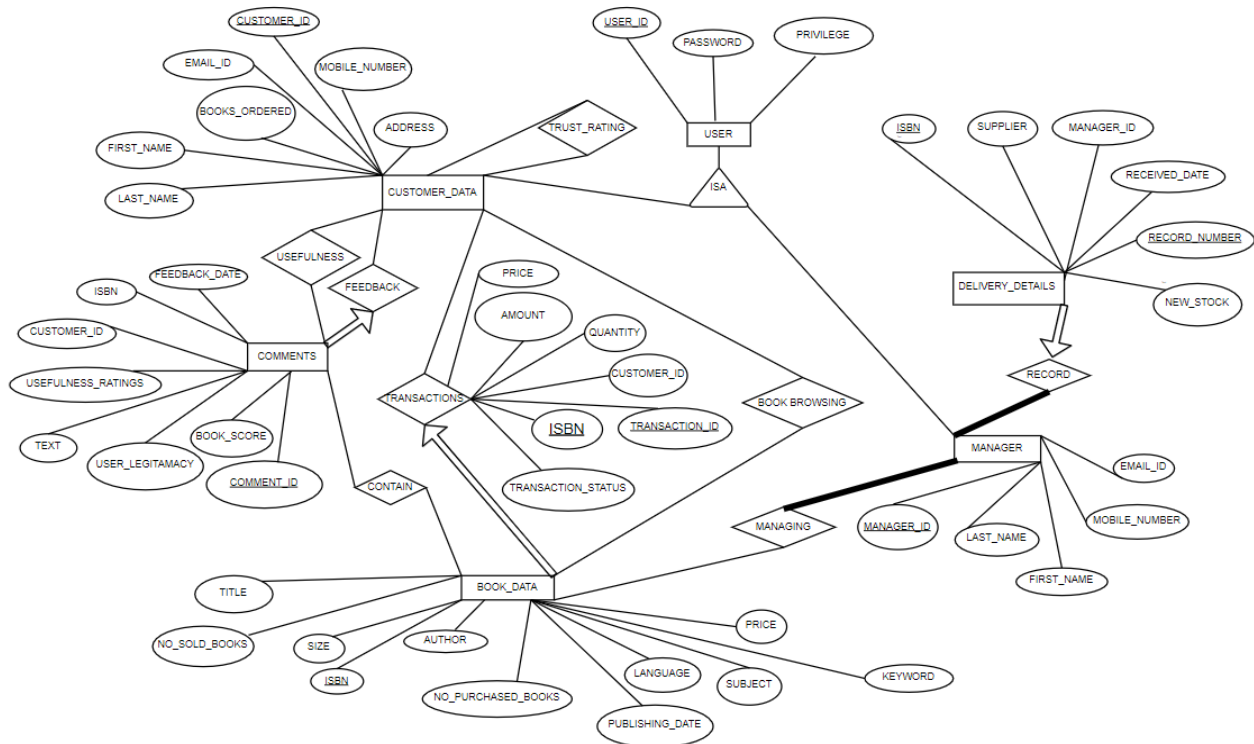


Figure 1: The ER-diagram of the project

1)CUSTOMER_DATA

In this Entity, the attributes are FIRST_NAME, LAST_NAME, CUSTOMER_ID, ROLE, ADDRESS, MOBILE_NUMBER, EMAIL_ID, BOOKS_ORDERED.

The primary key for this entity is CUSTOMER_ID and this entity is involved in several relationships. There are no foreign keys.

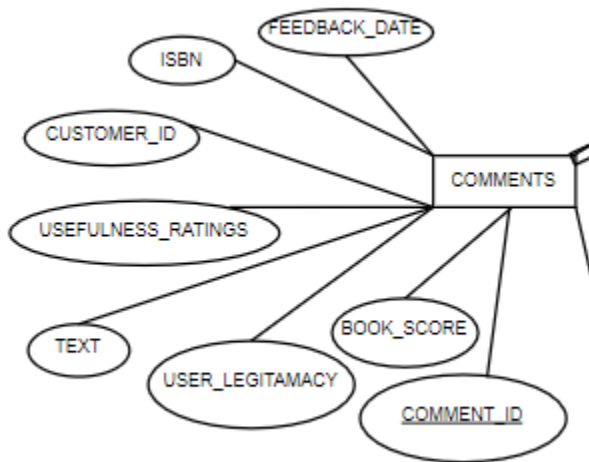
The customers are allowed to make comments on any book. Since only one comment per user per book is allowed, every user can make comments on many books, but every comment can be made by only one customer. This forms a one-to-many relationship and the relationship is named as FEEDBACK. This relationship is between the CUSTOMER_DATA and COMMENTS entities

In the USEFULNESS function, A user can provide a usefulness rating only once for a particular customer's comment but is allowed to change his/her usefulness rating for the particular customer's comment.

Users can access a comment and give it a usefulness score in three levels, namely, useless, useful, and very useful. A user will not be allowed to provide a usefulness rating for his/her own comments. We therefore named this relationship as USEFULNESS. This relationship is between the CUSTOMER_DATA and COMMENTS entities

Another relationship is the TRUST_RATING relationship where a user may declare zero or more other users as trusted or Not-trusted. Hence this relationship exists between the entity and itself.

2)COMMENTS:

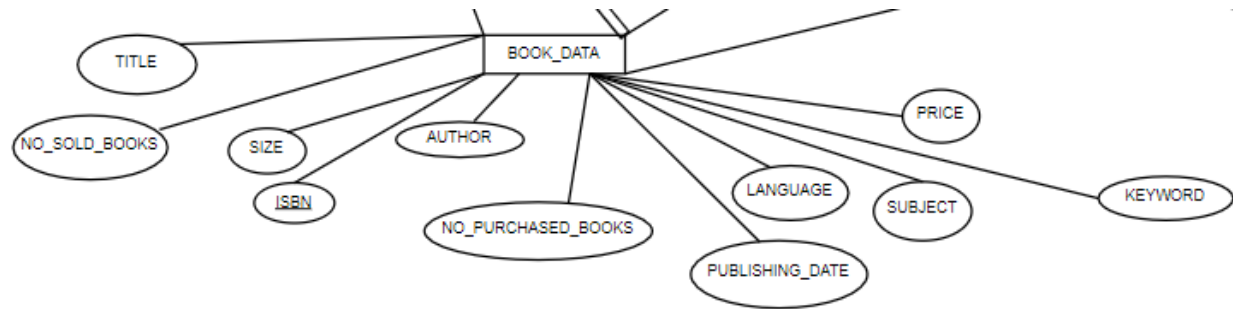


In this Entity, the attributes are ISBN, CUSTOMER_ID, COMMENT_ID, BOOK_SCORE, COMMENT_TEXT, USEFULNESS_RATINGS, USER_LEGITAMACY, FEEDBACK DATE.

The COMMENT_ID is the primary key for this entity. ISBN, CUSTOMER_ID are foreign keys.

The relationships for this entity are FEEDBACK, USEFULNESS and CONTAINS.

3)BOOK DATA:



In this Entity, the attributes are ISBN, TITLE, AUTHOR, PUBLISHER, LANGUAGE, PUBLISHING_DATE, EDITION_NUMBER, SIZE, PRICE, KEYWORD, SUBJECT, NO_PURCHASED_BOOKS, NO_SOLD_BOOKS.

The ISBN is the primary key for this entity. There is no foreign key.

The main relationship between BOOK_DATA and CUSTOMER_DATA is TRANSACTIONS.

The relationship attributes are CUSTOMER_ID, TRANSACTION_ID, TRANSACTION_DATE, ISBN, QUANTITY, TRANSACTION_STATUS, FINAL_AMOUNT

The ISBN and TRANSACTION_ID are the primary key for this entity. Price and Customer_id are the foreign keys of this relationship.

Whenever a customer makes an order of books, they can purchase multiple books of different quantities and are also allowed to make multiple transactions. On the other hand, every transaction has a single customer/ buyer involved.

This forms a many-to-one relationship between BOOK_DATA and CUSTOMER_DATA.

One of the functionalities which users may take advantage of is the book browsing function in which a User may search for books by asking conjunctive queries on authors, and/or publisher, and/or title words, and/or language. The system should also allow the user to specify that the results are to be sorted:

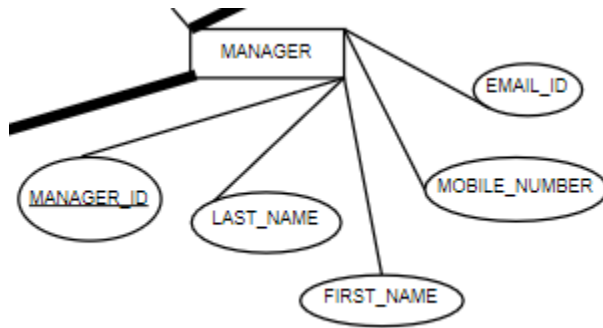
- (a) by publish date, or
- (b) by the average numerical score of the comments, or
- (c) by the average numerical score of the trusted user feedbacks.

A minimum of one input among AUTHOR, PUBLISHER, KEYWORD, LANGUAGE, SUBJECT is necessary, and the sorting order is also necessary in order to search based on the category and It is searched using the BOOK DATA table.

The output will be the ISBN, BOOK_NAME from BOOK_DATA table of the list of books that fall under the same category.

Therefore, a relationship exists between BOOK_DATA and CUSTOMER_DATA and is named as BOOK BROWSING.

4) MANAGER:



In this entity, the attributes are FIRST_NAME, LAST_NAME, MANAGER_ID, MOBILE_NUMBER, EMAIL_ID.

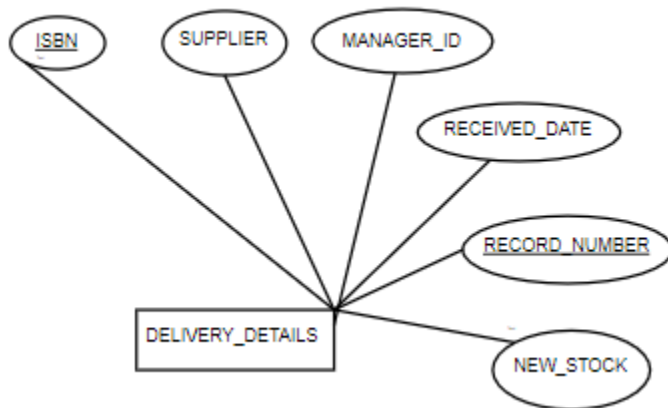
The primary key is MANAGER_ID. There is no foreign key.

There relationship called MANAGING exists between MANAGER and BOOK_DATA entities. In this relationship, functionalities like New book, Book statistics, Arrival of new copies, restock and delete book is possible only if this relationship exists.

In new book, the store managers may record the details of a new book and the number of books that have arrived in the warehouse. Therefore, the manager will store details like ISBN, TITLE, AUTHOR, PUBLISHER, LANGUAGE, PUBLISHING_DATE, EDITION_NUMBER, SIZE, PRICE, KEYWORD, SUBJECT, NO_PURCHASED_BOOKS, NO_SOLD_BOOKS.

In Arrival of new copies, restock and delete, the store manager should also again update attributes in the BOOK_DATA .

5) DELIVERY_DETAILS



In this entity, the attributes are RECORD_NUMBER, RECEIVED_DATE, MANAGER_ID, ISBN, NEW_STOCK, SUPPLIER

RECORD_NUMBER and ISBN together form the primary key for this table.

The main relationship for this table is the RECORD relationship. When a supplier provides more copies of a book, a manager must be present to note down details of the delivery and update other tables appropriately. Since many supplies can be made and only one manager is required to record for each delivery, there exists a Many-to-one relationship between DELIVERY_DETAILS and MANAGER entities.

TASK 3:

TECHNOLOGY SURVEY

Angular

Angular provides developers with the tools needed to build and structure large-scale JavaScript applications.

Besides that, Angular has some big advantages over some alternatives. For example, Angular is built and supported by Google engineers.

Alongside with these engineers, there is a huge community ready to help you with issues when the time comes.

Angular has more features out of the box than React.

Angular uses real DOM for its applications. For single-pages that get updated occasionally, real DOM is perfect. Also, the size of the library is huge, which makes the apps using Angular slower. For example, UpWork is one of the most popular apps that use Angular, as the need for the frequent update of the user feed is low.

Document Object Management (DOM) is an important metric in the performance evaluation of each of the frameworks.

As we have already discussed, the DOM organizes the HTML, XML documents in the form of a tree.

In Angular apps, a real DOM is used,

Compodoc is a static documentation generator for Angular. Similar to other documentation generators, it can create static HTML documentation based on the TSDoc comments in your code. Compodoc, however, comes with convenient features specifically for Angular, such as browsing your module structure, routes, and classifying classes into components, services, and so on.

The Angular framework uses a two-way binding.

approach that changes the model state automatically when any change is made in the UI element and vice versa. The React library uses a one-way approach. This approach allows the change in the UI elements only after a change is made in the model state. The two-way approach seems to be easier, but

Angular 2 uses both one- and two-way data binding:

changing data impacts view and changing view triggers changes in data.

Advantages using Angular

dependency injection and Powerful Router

Angular Router has built-in support for lazy loading components, so the modules are loaded from the server as the user navigates within the app. Huge apps have a lot of benefits because of this feature

cross-platform mobile applications

Angular was the first framework that was supported by the mobile applications

It allows building cross-platform mobile applications for Android and iOS

Availability of packages

There is a huge repository of open-source packages available for Angular developers.

Disadvantages using Angular

Backward Compatibility

Angular impossible to jump from Angular 2.0 to 7.0. First, you need to install updates between the versions, one by one.

Steeper learning curve

little bit uncomfortable with the steep learning curve that it has

optimization

It is a disadvantage when it comes to search engine optimization.

Too many versions

the current version having with a total of 6 major releases in between. This can cause confusion especially for beginners.

React

React is developed and maintained by Facebook and used in their products, including Instagram and WhatsApp. It has been around for around since 2013, so it's not exactly new. It's also one of the most popular projects on GitHub, with more than 150,000 stars at the time of writing. Some of the other notable companies using React are Airbnb, Uber, Netflix, Dropbox, and Atlassian. Sounds good to me.

React uses a virtual DOM and is best suited for pages that need regular content updates. The size of the library is small too, which makes it more lightweight and dynamic. For example, Instagram uses React as it needs the user feed to update frequently.

when designing a React app developers often nest child components within higher-order parent components.

One-way binding makes the code more stable, and also makes debugging an app build with React versus Angular app much easier. Still, the one/two-way binding of Angular is simpler to work with and makes the framework more flexible.

Hooks have been introduced to make functional components simpler and more composable.

You can now split large functions into smaller atomic parts, allowing you to divide up related pieces of functionality

React is a library, and it has full backward compatibility. You can add different releases of the libraries to the application and update the old ones.

Advantages using React

instead of classic templates, it has JSX, an XML-like language built on top of JavaScript

class-based components with lifecycle hooks or simpler functional components

state management using `setState` and hooks.

XSS protection

code splitting and lazy loading

error handling boundaries

utilities for unit-testing components

Disadvantages using React

Poor documentation

The problem with documentation traces back to constant releases of new tools.

Different and new libraries like Redux and Reflux are promising to accelerate the work of a library or improve the entire React ecosystem

HTML in my JavaScript! – JSX as a barrier

Developers and designers complain about JSX's complexity and consequent hard learning curve.

Additional SEO problems

There have been concerns that Google and other search engines can't index or poorly index dynamic web pages with client-side rendering.

When we compare Angular and React, I believe that Angular would be much more suitable for this project.

Application Logic

Flask

Flask is Python micro-framework for building web applications that's known to be light and scalable

Flask has its strengths. Flask was built to be scalable and simple to get started with.

Applications built with Flask are clearly lighter when compared to Django counterparts.

As such, Python developers usually refer to Flask as a microframework.

SQLAlchemy is a Python ORM (Object Relational Mapper) and SQL toolkit that allows Python developers to use SQL-based databases

(for data persistence and retrieval) without knowing or dealing with SQL but Python objects and methods instead.

URI is most regularly than not set by the see decorator and centralized setup is additionally conceivable. Some time recently the designs are coordinated with the URIs, the last mentioned is sorted in a default arrange

Flask has many features such as:

Urls.py is utilized to set the association properties and demands are handled by the primary coordinating see of regex list

Extend Layout is Arbitrary structure

Flask gives straightforwardness, adaptability and fine-grained control. It is unopinionated

Advantages using Flask

It's easy to setup and get started with

It's has an active community

It has a good documentation

It's lighter than Django and may constitute a good lightweight alternative to Django for many projects

It can be easily extended.

Disadvantages using Flask

Fewer Seasoned Developers

Lack of True Multiprocessor Support

Speed Limitations

Not the Go-To Language for Mobile App Development

Not Ideal for Memory-Intensive Tasks

Design Restrictions

Django

Django is an open-source python web framework used for rapid development, pragmatic, maintainable, clean design, and secure websites.

A web application framework is a toolkit of all components needed for application development.

The main goal of the Django framework is to allow developers to focus on components of the application that are new instead of spending time on already developed components.

Django is fully featured than many other frameworks on the market. It takes care of a lot of hassles involved in web development; enables users to focus on developing components needed for their application.

Urls.py is utilized to set the association properties and demands are handled by the primary coordinating see of regex list

Extend Layout is Conventional extend structure

Django gives an all-inclusive encounter: you get an admin board, database interfacing, an ORM, and registry structure for your apps and ventures out of the box.

With Django, We can build applications much more quickly without sacrificing features. The developer experience is also quite good – and not just because Python is awesome. It's also because there are good debugging tools, logging is already set up, and there's a server that automatically restarts with file changes.

Advantages using Django

1. Easy to Use
2. It's fast and simple
3. Excellent Documentation for real-world application
4. It's secure
5. It suits any web application project

Disadvantages using Django

Django ORM, created before SQLAlchemy existed, is now much inferior to SQLAlchemy.

Django is large and is considered monolithic piece of software.

This allows the community to develop hundreds of reusable modules and apps but has also limited the speed of development of the Django.

When we compare Flask and Django , I find Flask much more preferable and I am also comfortable using flask since I have good knowledge in Python after taking courses CMPSC 131, 132 and CMPSC 465.

DATA STORAGE.

MySQL

MySQL is an open-source relational database management system. As with other relational databases, MySQL stores data in tables made up of rows and columns. Users can define, manipulate, control, and query data using Structured Query Language, more commonly known as SQL

it's used to store and retrieve data in a wide variety of popular applications, websites, and services.

Advantages using MySQL

MySQL is a Relational Database Management System or RDBMS which means that it stores and presents data in tabular form, organized in rows and columns.

It is more secure as it consists of a solid data security layer to protect sensitive data from intruders and passwords in MySQL are encrypted.

It is available for free to download and use from the official site of MySQL.

Compatible with most of the operating systems, including Windows, Linux, NetWare, Novell, Solaris and other variations of UNIX.

Provides the facility to run the clients and the server on the same computer or on different computers, via internet or local network.

It has a unique storage engine architecture which makes it faster, cheaper and more reliable.

Disadvantages using MySQL

MySQL is not very efficient in handling very large databases.

It doesn't have as good a developing and debugging tool as compared to paid databases.

Versions less than 5.0 do not support COMMIT, stored procedure and ROLE.

It is prone to data corruption as it inefficient in handling transactions.

It does not support SQL check constraints.

MongoDB

MongoDB stores data in flexible, JSON-like documents, meaning fields can vary from document to document and data structure can be changed over time

The document model maps to the objects in your application code, making data easy to work with. Ad hoc queries, indexing, and real time aggregation provide powerful ways to access and analyze application data. MongoDB is a distributed database at its core, so high availability, horizontal scaling, and geographic distribution are built in and easy to use

Advantages using MongoDB

Performance

By storing a majority of the data in RAM, query performance in MongoDB is much quicker.

Simplicity

Because of its expressive query language, many users find MongoDB query syntax to be rather simple and much easier to understand than SQL

Flexibility

MongoDB has no predefined schema and thus has very dynamic schematics architecture for unstructured data and storage options.

Scalability

One of the most attractive features of MongoDB is its reliable scalability. As opposed to SQL databases that utilize vertical scalability, MongoDB (like other NoSQL databases) uses shards for horizontal scalability.

Disadvantages using MongoDB

Transactions

One of the downsides of MongoDB is that it doesn't support transactions. Though fewer and fewer applications are requiring transactions, there are still some that need transactions in order to update multiple documents/collections.

Joins

Joining documents in MongoDB is no easy task and though 3.2 introduced left-outer joins, developers are still working on the function

and it's not quite mature yet.

Indexing

As mentioned earlier, enjoying MongoDB's quick speeds and high performance is only possible with the right indexes.

Duplicates

Some of these downsides could ultimately lead to duplicate data (as has happened for many users in the past).

Since MySQL is a relational database engine, I prefer MySQL is my favorable option because the transaction functionality is only available in RDBMs (Relational Databases).

TASK 4:**LOGICAL DATABASE DESIGN AND NORMALIZATION**

In this section, we are going to create schema for our project.

customer_id	first_name	last_name	Address	Mobile number	email_id	Book ordered
101	Peter	smith	Ashburn VA	3017641212	donnt@mail	15
102	John	nickerson	Chantilly VA	7031256548	mymail@gmail.com	20

In CUSTOMER_DATA, we do not need to perform Normalization because the Functional Dependency is good and CUSTOMER_ID is the primary key. All the non-key attributes is decided by key-attributes.

Therefore, CUSTOMER_DATA is already in Boyce Codd Normal Form.

ISBN	Title	Author	Publisher	language	publish date	edition number	size	price	keyword	subject	no_purchased_books	no_sold_books
IS202	Java	John	sc001	English	01-02-92	1	320	23.50	computer	comp	200	120
IS404	Puppet	Peter	pt0020	English	08-06-98	3	260	49.60	computer	comp	260	70

In BOOK_DATA, we again do not have to perform Normalization because the Functional Dependency is good and ISBN is the primary key.

Therefore, BOOK_DATA is in Boyce Codd Normal Form.

In TRANSACTIONS, ISBN and PRICE are the primary keys and data redundancy occurs and the functional dependency is bad.

Using the BCNF decomposition algorithm, we split TRANSACTIONS into TRANSACTION_HEADER and TRANSACTION_DETAIL tables.

In the TRANSACTION_DETAIL table we have transaction_id, ISBN, quantity, status, price and amount. The primary keys are transaction_id, isbn and foreign keys are transaction_id.

transaction_id	isbn	quantity	status	price	amount
6021	IS202	1	open	23.50	23.50
6021	IS404	1	open	49.60	49.60

In the TRANSACTION_HEADER table, we have transaction_id, transaction_date, customer_id.

The primary keys are transaction_id and the foreign keys are customer_id.

transaction_id	transaction_date	customer_id
6021	12-02-2020	101
6022	12-03-2020	102

In COMMENTS, the USEFULNESS_RATINGS, USER_LEGITAMACY cause a lot of data redundancy and the functional dependency is bad.

Using the BCNF decomposition algorithm, we split COMMENTS into COMMENTS, usefulness_rate and customer_trust_rate.

In COMMENTS, the primary key is comment_id and the foreign keys are ISBN and customer_id.

comment_id	ISBN	customer_id	book_score	comment_text	feedback_date
800021	IS202	101	10	a...n	03-02-2021
800022	IS202	102	10	a...n	03-03-2021

In usefulness_rate the primary key are customer_id, comment_id and the foreign key is comment_id

customer_id	comment_id	usefulness_ratings
105	800021	1
122	800022	2

in customer_trust_rate, the primary key are customer_id, rated_customer_id and the foreign key is rated_customer_id

customer_id	rated_customer_id	user_legitimacy
305	105	0

2018	106	1
------	-----	---

In DELIVERY_DETAILS, ISBN and RECORD_NUMBER are the primary keys and data redundancy occurs and the functional dependency is bad.

record_number	ISBN	new_stock
921501	IS404	20
921501	IS98562	30

Using the BCNF decomposition algorithm, we split DELIVERY_DETAILS into DELIVERY_DETAILS and delivery_header tables.

In the DELIVERY_DETAILS, the primary key is RECORD_NUMBER, ISBN and the foreign key is ISBN

In the delivery_header, the primary key is RECORD_NUMBER and the foreign key is manager_id.

manager_id	first_name	last_name	Mobile number	email_id
2001	Scott	Tiger	7045256548	mya@gmail.com
2002	Tom	jackson	798256548	anothr@gmail.com

In MANAGER, we do not need to perform Normalization because the Functional Dependency is good and MANAGER_ID is the primary key. All the non-key attributes are decided by key-attributes.

Below is the final schema after Normalization in Boyce Codd Normal Form:

```
CREATE TABLE customer_data (
  customer_id   varchar(8) NOT NULL,
  first_name    varchar(30) NOT NULL,
  last_name     varchar(30) NOT NULL,
  address       varchar(150) NOT NULL,
  mobile_number int       NOT NULL,
  email_id      varchar(50) NOT NULL,
  book_ordered  varchar(8) NOT NULL,
  PRIMARY KEY (customer_id)
);
```

```
CREATE TABLE manager (
  manager_id   varchar(8) NOT NULL,
  first_name    varchar(30) NOT NULL,
  last_name     varchar(30) NOT NULL,
  mobile_number int       NOT NULL,
  email_id      varchar(50) NOT NULL,
  PRIMARY KEY (manager_id)
);
```

```
CREATE TABLE application_users (
  user_id       varchar(8) NOT NULL,
  user_password varchar(30) NOT NULL,
  privilege     int
);
```

```
CREATE TABLE book_data (
  isbn          varchar(13) NOT NULL,
  title          varchar(50) NOT NULL,
  author         varchar(60) NOT NULL,
  publisher       varchar(60) NOT NULL,
  language       varchar(20) NOT NULL,
  publishing_date varchar(20) NOT NULL,
  edition_number int       NOT NULL,
  size           int       NOT NULL,
  price          DECIMAL(5,2) NOT NULL,
  keyword         varchar(140) NOT NULL,
  subject        varchar(140) NOT NULL,
  no_purchased_books int,
  no_sold_books  int,
  PRIMARY KEY (isbn)
);
```

```
CREATE TABLE transaction_header (
  transaction_id int NOT NULL,
  transaction_date DATE NOT NULL,
  customer_id varchar(8) NOT NULL,
  PRIMARY KEY (transaction_id),
  FOREIGN KEY (customer_id) REFERENCES customer_data(customer_id)
);
```

```
CREATE TABLE transaction_detail (
  transaction_id int NOT NULL,
  isbn varchar(13) NOT NULL,
  quantity int NOT NULL,
  status varchar(15) NOT NULL,
  price DECIMAL(5,2) NOT NULL,
  amount DECIMAL(15,2) NOT NULL,
  PRIMARY KEY (transaction_id, isbn),
  FOREIGN KEY (transaction_id) REFERENCES transaction_header(transaction_id)
)
```

```
CREATE TABLE comments (
  comment_id int NOT NULL,
  isbn varchar(13) NOT NULL,
  customer_id varchar(8) NOT NULL,
  book_score int NOT NULL,
  comment_text varchar(100) NOT NULL,
  feedback_date DATE NOT NULL,
  PRIMARY KEY (comment_id),
  FOREIGN KEY (isbn) REFERENCES book_data(isbn),
  FOREIGN KEY (customer_id) REFERENCES customer_data(customer_id)
);
```

```
CREATE TABLE usefulness_rate (
  customer_id varchar(8) NOT NULL,
  comment_id int NOT NULL,
  usefulness_ratings int NOT NULL,
  PRIMARY KEY (customer_id, comment_id),
  FOREIGN KEY (comment_id) REFERENCES comments(comment_id)
);
```

```
CREATE TABLE customer_trust_rate (
  customer_id varchar(8) NOT NULL,
  rated_customer_id varchar(8) NOT NULL,
  user_legitimacy int NOT NULL,
```

```
PRIMARY KEY (customer_id, rated_customer_id),  
FOREIGN KEY (customer_id) REFERENCES customer_data(customer_id),  
FOREIGN KEY (rated_customer_id) REFERENCES customer_data(customer_id)  
);
```

```
CREATE TABLE delivery_header (  
    record_number int NOT NULL,  
    received_date DATE NOT NULL,  
    manager_id varchar(8) NOT NULL,  
    supplier varchar(100) NOT NULL,  
    PRIMARY KEY (record_number),  
    FOREIGN KEY (manager_id) REFERENCES manager(manager_id)  
);
```

```
CREATE TABLE delivery_details (  
    record_number int NOT NULL,  
    isbn varchar(13) NOT NULL,  
    new_stock int NOT NULL,  
    PRIMARY KEY (record_number, isbn),  
    FOREIGN KEY (record_number) REFERENCES delivery_header(record_number),  
    FOREIGN KEY (isbn) REFERENCES book_data(isbn)  
);
```

REFERENCES:

<https://www.sitepoint.com/react-vs-angular/>

<https://www.guru99.com/react-vs-angular-key-difference.html#7>

<https://academind.com/tutorials/angular-vs-react-vs-vue-my-thoughts/>

<https://www.geeksforgeeks.org/differences-between-django-vs-flask/>

<https://www.w3spoint.com/mysql-advantages-disadvantages>

CMPSC 431W Project Guideline